

resolve (runner patch) – guide

Revision: 1 Last modified: 2026-09-26T14:10:00Z

Source: `constitution/scripts/codegraph/runner_patches/resolve2.py` (63 lines). A **module driven by** `fk_index_patch.py`, not a command. Citations are `resolve2.py:<line>` unless another file is named. Machinery guide: [fk_index_patch](#).

Overview

Stock CodeGraph 1.6.0 `frameworks/swift-objc.js` `buildObjcMap()`, reached the first time a Swift `calls` reference survives to the framework stage, does `context.getNodesByKind('method').filter((n) => n.language === 'objc')`. `getNodesByKind` runs `SELECT * FROM nodes WHERE kind = ?` with `.all()`, maps every row to a node and **pins the array** in the per-resolver `nodesByKindCache` for the rest of the pass. On the ATMOSphere index that is 3,478,494 method nodes (46,158 objc), and the single call exhausts the default 4,288 MB V8 heap (`FATAL "Reached heap limit"`); every resolver pool worker that meets a Swift ref dies the same way (docstring lines 3-15, citing ATM-1030 evidence files named in the source; those files were not re-read for this guide).

Fix (one anchor, lines 38-48): iterate `context.iterateNodesByKind('method')` (same SQL text and row mapping, lazy, same statement order) and keep only `language === 'objc'` rows. Memory becomes $O(\#objc \text{ methods})$ and nothing is pinned in `nodesByKindCache`. Filter and element order are unchanged, so `candidates[0]` (the chosen bridge target) is unchanged (lines 21-24).

Prerequisites

Reads `lib/dist/resolution/frameworks/swift-objc.js` and reads (never edits) `lib/dist/resolution/index.js`. Python stdlib plus `runner_patches/common.py`. Default-set patch (`runner_patches/__init__.py:17`), applied fifth.

Usage

Only through `fk_index_patch.py` :

```
python3 scripts/codegraph/fk_index_patch.py --patches resolve2 --dst <scratch dir>
```

- VERIFIED (ran: `tests/test_resolve2.sh` with `TMPDIR` set to a scratch directory; saw: SUMMARY cases=3 failed=0, exit 0) . The test builds a `--patches resolve2` runner into its own workdir with `HELIX_CODEGRAPH_RUNNER_DIR` set to that workdir (`test_resolve2.sh:39`), never the real cache.
- VERIFIED (ran: `resolve2.plan` via `fk_index_patch.plan_all` against stock 1.6.0; saw: APPLIED, file `lib/dist/resolution/frameworks/swift-objc.js`, anchor "`buildObjcMap`
`getNodesByKind('method').filter(objc)`", detail `order_preserved true`) .
- NOT RUN : the optional Q4 check (`REAL=<db> REAL_ROOT=<root> REAL_OFF=<rowid>`) opens a real index read-only and was deliberately not run (hard rule: never touch the live index).

Behaviour (source-traced)

`plan(view)` (lines 51-63):

1. Refuse `resolve2`: source is ALREADY patched if `helix-resolve2` occurs in `swift-objc.js` (line 53-54).

2. `Refuse` resolution context lacks the expected `iterateNodesByKind` member (`index.js` shape changed) unless `resolution/index.js` contains the line `iterateNodesByKind: (kind) =>`
`this.queries.iterateNodesByKind(kind),` exactly once (lines 36, 55-57). This is the fail-closed guard against a silent fallback to the materializing path.
3. One `replace_once` of the
`context.getNodesByKind('method').filter(...)` expression by a
`for...of` over `context.iterateNodesByKind('method')` pushing objc rows into an array (lines 38-48, 59). There is no `NOT_NEEDED` path in this patch.

3 1 5

What the tests prove (`tests/test_resolve2.sh` , header lines -)

Q1 after driving the bridge `queries.getNodesByKind('method')` was not called and `nodesByKindCache` holds no `method` ; Q2 bridge targets are identical to the stock package's, order-sensitive (three `.m` files define each selector, plus same-named non-objc Java methods indexed first); Q3 a full-index sorted edge+ref dump is byte-identical to stock; Q4 (real index, opt-in only) 200 real pending refs resolve under the default heap without OOM. `tests/test_resolve2_mutations.sh` applies single-edit mutants (table in `test_resolve2_mutate.py`) and requires the named tests red.

Edge cases

- Already-patched input: `VERIFIED` (parent guide: plan on the patched text -> `Refuse "source is ALREADY patched"`) .
- Anchor removed or duplicated: `VERIFIED` (parent guide: `Refuse "anchor ... occurs 0x|2x ... expected exactly 1"`) .
- `iterateNodesByKind` missing from the resolution context: `Refuse` by the guard above (source-traced; not separately run).
- The patched loop still builds an array of all objc methods (46,158 on the ATMOSPHERE index per the source), so memory is bounded by the objc method count, not by zero.

Related scripts

[fk_index_patch](#), siblings [fkidx1](#), [lockfix1](#), [datafrag1](#), [resolve1](#) (edits the `resolution/index.js` that `resolve2` only reads), [mcpro1](#), [codegraph_safe](#).

Last verified

2026-09-26: `tests/test_resolve2.sh` 3/3 PASS and `tests/test_resolve2_mutations.sh` SUMMARY cases=6 failed=0 (golden plus 5 mutants killed), both with a scratch `TMPDIR` ; the real runner cache directory listing was unchanged.